

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Distributed Object Storage & Fault Tolerance Testing

Data _____ Class 24 Applied Computing Name Zhou Jiaqi ID 202483910036

1. Experimental Purpose

In this experiment I learned how an object storage system is different from an ordinary file-path based storage system. Instead of treating a file only as something stored under one local directory, I used a bucket/object structure similar to S3-compatible storage. I also tested what happens when one backend storage directory is removed, so I could see the fault-tolerance behavior rather than only reading about it.

My specific goals were to deploy a small distributed MinIO cluster, upload a large object, check how the object was represented in the backend storage folders, simulate one disk failure, and verify whether the object could still be listed after the failure.

2. Experimental Environment

Item	My environment / setting
Platform	GitHub Codespaces with VS Code in the browser
Container runtime	Docker and Docker Compose
Object storage software	MinIO server image: quay.io/minio/minio
Client tool	MinIO Client image: minio/mc
Simulated storage disks	data1, data2, data3, and data4 directories
Test object	bigfile.bin, 20 MiB random binary file

3. Detailed Experimental Procedure

Step 1 and Step 2: Create the lab directory and storage directories

I first created a separate working folder named minio-lab and entered it. Then I created four independent directories named data1, data2, data3, and data4. In this experiment these four directories act like four separate storage disks for the distributed MinIO service.

```
mkdir minio-lab && cd minio-lab
mkdir -p data1 data2 data3 data4
ls
```

```
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ mkdir minio-lab && cd minio-lab
@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ mkdir -p data1 data2 data3 data4
@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ ls
data1 data2 data3 data4
```

Figure 1. Creating the lab folder and four simulated disk directories.

Step 3: Create docker-compose.yml

Next, I created docker-compose.yml in the minio-lab folder. This file describes two services: the MinIO server container and the MinIO client container. I mapped the four local storage directories into the MinIO container so that MinIO could use them as four backend storage paths.

code docker-compose.yml

The main content of the file is shown below in simplified form:

```
services:
  minio:
    image: quay.io/minio/minio
    container_name: cloud-minio
    volumes:
      - ./data1:/data1
      - ./data2:/data2
      - ./data3:/data3
      - ./data4:/data4
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: password123
    command: server /data{1...4} --console-address ":9001"

  mc:
    image: minio/mc
    container_name: cloud-mc
    depends_on:
      - minio
    entrypoint: /bin/sh
    tty: true
```

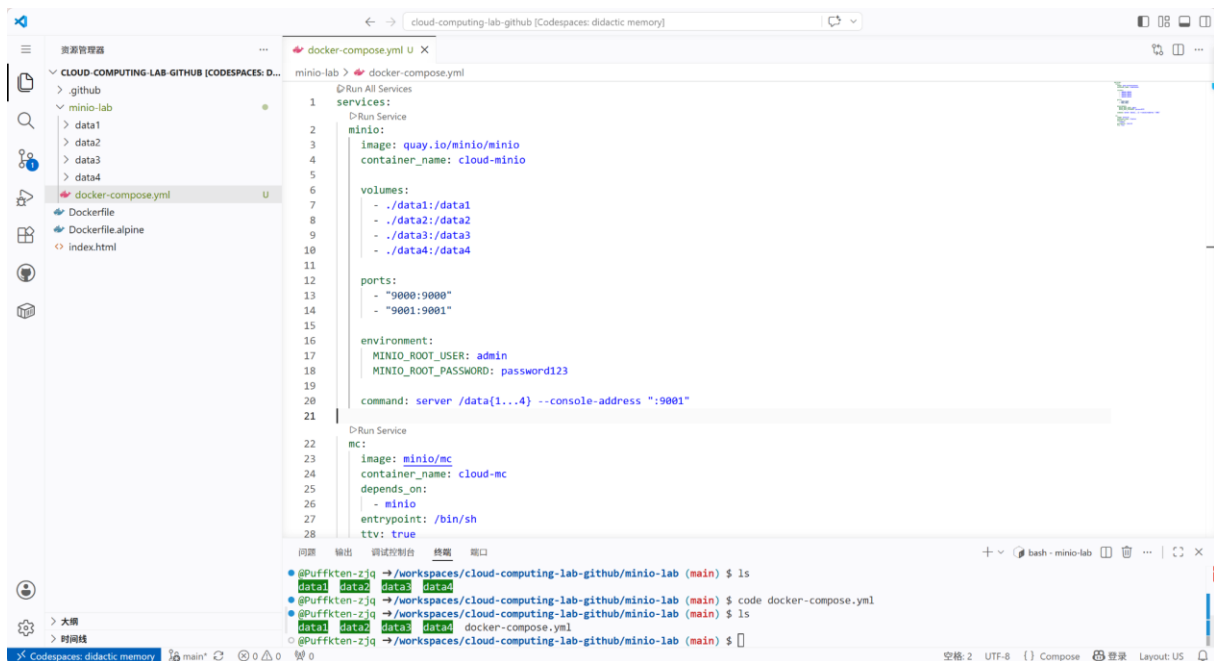


Figure 2. Creating and checking the docker-compose.yml file.

Step 4: Explain the docker-compose.yml file

Before starting the cluster, I checked each important part of the Compose file. The explanation is listed in the table below.

Component	Meaning in my experiment
image	The Docker image used to create each container. I used quay.io/minio/minio for the server and minio/mc for the client.
container_name	The fixed container name. This made later commands easier because I could directly use cloud-minio and cloud-mc.
volumes	The mapping between my local folders and the folders inside the MinIO container. These mappings simulated four independent storage disks.
ports	The exposed ports. Port 9000 is the S3-compatible API port, and port 9001 is the MinIO console port.
environment	The initial root username and password used to log in or configure the MinIO client.
command	The startup command for MinIO. server /data{1...4} tells MinIO to use the four storage paths as one distributed storage set.
depends_on	The mc client service is started after the minio service is created, so the client container can connect to the server container.
entrypoint and tty	These keep the mc container available as a lightweight shell container for running MinIO client commands.

Step 5: Launch the distributed cluster

After saving the Compose file, I started the services in detached mode. Docker pulled the required images, created the network, and started both containers.

```
docker compose up -d
docker ps
```

The docker ps result showed that cloud-minio and cloud-mc were both running. This meant the distributed MinIO cluster and the client container were ready for the next steps.

```
● @Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
[+] Running 16/16
  ✓ mc Pulled                                6.8s
  ✓ minio Pulled                             7.8s
[+] Running 3/3
  ✓ Network minio-lab_default Created         0.1s
  ✓ Container cloud-minio Started            4.8s
  ✓ Container cloud-mc Started              1.3s
● @Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
3b313150108f   minio/mc      "/bin/sh"               About a minute Up About a minute
8b19ab75a79b   quay.io/minio/minio "/usr/bin/docker-ent..." About a minute Up About a minute 0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
cloud-minio
```

Figure 3. Launching the MinIO cluster and verifying running containers.

Step 6: Configure the MinIO client alias

I then configured an alias named myminio inside the mc container. The alias points to the MinIO server through the internal Docker service name minio and port 9000. This is easier than typing the full endpoint every time.

```
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
```

The output showed “Added myminio successfully”, so the client could communicate with the server.

```
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added 'myminio' successfully.
```

Figure 4. Configuring the MinIO client alias.

Step 7: Create a bucket

After the alias was ready, I created a bucket named my-cloud-storage. In object storage, a bucket is the top-level container for objects, unlike a traditional local directory path.

```
docker exec cloud-mc mc mb myminio/my-cloud-storage
```

The command returned a successful bucket creation message.

```
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc mb myminio/my-cloud-storage
Bucket created successfully 'myminio/my-cloud-storage'.
```

Figure 5. Creating the cloud storage bucket.

Step 8: Create and upload a large test file

To make the storage layout visible, I generated a 20 MiB random binary file. I checked its size, copied it into the mc container, and uploaded it to the bucket. I used a larger file because a tiny file would not show the distributed object structure as clearly.

```
dd if=/dev/urandom of=bigfile.bin bs=1M count=20
ls -lh bigfile.bin
docker cp bigfile.bin cloud-mc:/bigfile.bin
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
```

The upload output showed that 20.00 MiB was transferred successfully.

```
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ dd if=/dev/urandom of=bigfile.bin bs=1M count=20
20+0 records in
20+0 records out
20971520 bytes (21 MB, 20 MiB) copied, 0.0886435 s, 237 MB/s
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -lh bigfile.bin
-rw-rw-rw- 1 codespace codespace 20M Jun 11 02:37 bigfile.bin
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker cp bigfile.bin cloud-mc:/bigfile.bin
Successfully copied 21MB to cloud-mc:/bigfile.bin
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
'/bigfile.bin' -> 'myminio/my-cloud-storage/bigfile.bin'
```

Total	Transferred	Duration	Speed
20.00 MiB	20.00 MiB	00m00s	29.47 MiB/s

Figure 6. Creating, verifying, copying, and uploading the 20 MB test file.

Step 9: Verify the object upload

I listed the bucket contents to confirm that the uploaded object was visible through the object storage interface.

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

The output showed bigfile.bin with a size of 20 MiB, so the upload was complete.

```
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-11 02:40:21 UTC] 20MiB STANDARD bigfile.bin
```

Figure 7. Verifying that bigfile.bin exists in the bucket.

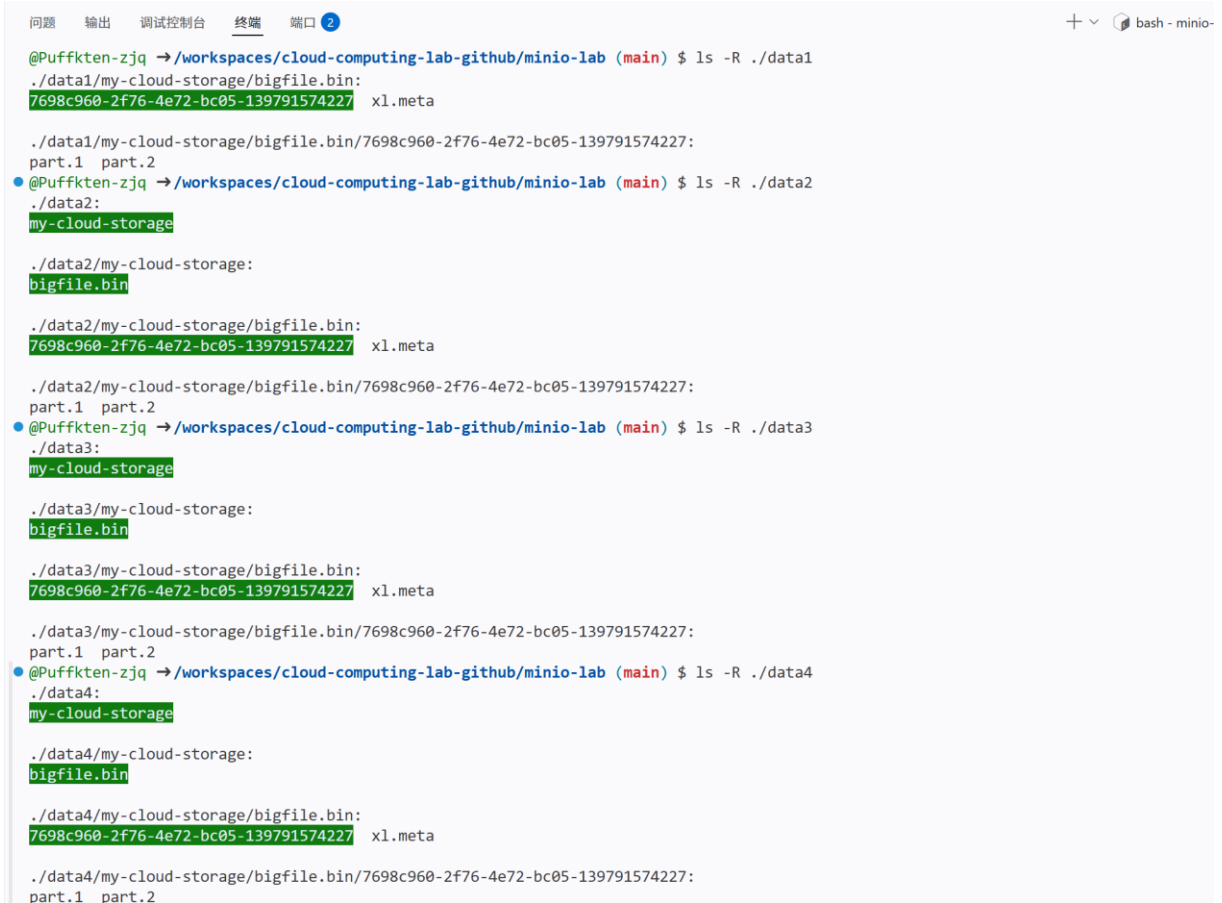
Step 10: Observe backend storage fragmentation

After the object was uploaded, I inspected the four backend storage directories. The purpose here was not to open the object as a normal file, but to see how MinIO stored its metadata and data fragments internally.

```
ls -R ./data1
ls -R ./data2
```

```
ls -R ./data3
ls -R ./data4
```

Each storage directory contained a bucket folder and an object folder for bigfile.bin. Inside the object directory, I could see xl.meta and part files such as part.1 and part.2. This showed that MinIO did not simply copy one complete object into one local directory. Instead, it placed metadata and object parts across the four storage locations.



```
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data1
./data1/my-cloud-storage/bigfile.bin:
7698c960-2f76-4e72-bc05-139791574227 xl.meta

./data1/my-cloud-storage/bigfile.bin/7698c960-2f76-4e72-bc05-139791574227:
part.1 part.2
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data2
./data2:
my-cloud-storage

./data2/my-cloud-storage:
bigfile.bin

./data2/my-cloud-storage/bigfile.bin:
7698c960-2f76-4e72-bc05-139791574227 xl.meta

./data2/my-cloud-storage/bigfile.bin/7698c960-2f76-4e72-bc05-139791574227:
part.1 part.2
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data3
./data3:
my-cloud-storage

./data3/my-cloud-storage:
bigfile.bin

./data3/my-cloud-storage/bigfile.bin:
7698c960-2f76-4e72-bc05-139791574227 xl.meta

./data3/my-cloud-storage/bigfile.bin/7698c960-2f76-4e72-bc05-139791574227:
part.1 part.2
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data4
./data4:
my-cloud-storage

./data4/my-cloud-storage:
bigfile.bin

./data4/my-cloud-storage/bigfile.bin:
7698c960-2f76-4e72-bc05-139791574227 xl.meta

./data4/my-cloud-storage/bigfile.bin/7698c960-2f76-4e72-bc05-139791574227:
part.1 part.2
```

Figure 8. Observing the backend storage structure in data1-data4.

Step 11: Simulate one disk failure

To simulate a disk failure, I stopped the cluster, deleted the data3 directory, recreated an empty data3 directory, and then restarted the cluster. This is similar to replacing a broken disk with a blank disk in a small storage system.

```
docker compose down
sudo rm -rf data3
mkdir data3
docker compose up -d
docker ps
```

After restarting, docker ps again showed that both cloud-minio and cloud-mc were running.

```

@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose down
[+] Running 3/3
✓ Container cloud-mc Removed 10.2s
✓ Container cloud-minio Removed 0.2s
✓ Network minio-lab_default Removed 0.1s
@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ sudo rm -rf data3
@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ mkdir data3
@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
[+] Running 3/3
✓ Network minio-lab_default Created 0.1s
✓ Container cloud-minio Started 0.4s
✓ Container cloud-mc Started 0.8s
@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
46cc1cf69bf2	minio/mc	"/bin/sh"	About a minute ago	Up About a minute	
f75aaf295494	quay.io/minio/minio	"/usr/bin/docker-ent..."	About a minute ago	Up About a minute	0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp

Figure 9. Simulating disk failure by deleting data3 and restarting the cluster.

Step 12: Verify fault tolerance

After the cluster restart, I listed the bucket again. At first I got an error because the mc container had been recreated and the alias was not available in the new container. I also had to make sure that I used myminio/my-cloud-storage without adding a leading slash. I fixed this by setting the alias again and then running the list command.

```

docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
docker exec cloud-mc mc ls myminio/my-cloud-storage/

```

The final output still showed bigfile.bin. This confirmed that deleting one backend storage directory did not make the object disappear from the object storage interface.

```

@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio/my-cloud-storage/
mc: <ERROR> Unable to list folder. Requested path '/myminio/my-cloud-storage' not found
@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added 'myminio' successfully.
@Puffkten-zjq →/workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-11 02:40:21 UTC] 20MiB STANDARD bigfile.bin

```

Figure 10. Reconfiguring the client alias and confirming the object is still accessible.

Step 13: Save changes with Git

Finally, I saved the lab work with Git. I first tried git add minio-lab while I was already inside the minio-lab directory, so Git could not find that path. I solved it by moving back to the project root directory and then adding the minio-lab folder from there.

```

git status
git add minio-lab
cd ..
git add minio-lab
git commit -m "Complete MinIO distributed storage lab"
git push

```

The commit was created successfully and pushed to the remote GitHub repository. The screenshots also show that the upload included the Docker Compose file and the generated MinIO storage files.


```

@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  ./

nothing added to commit but untracked files present (use "git add" to track)
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git add minio-lab
fatal: pathspec 'minio-lab' did not match any files
@Puffkten-zjq → /workspaces/cloud-computing-lab-github/minio-lab (main) $ cd ..
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ git add minio-lab
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ git commit -m "Complete MinIO distributed storage lab"
[main b408693] Complete MinIO distributed storage lab
131 files changed, 32 insertions(+)
create mode 100644 minio-lab/bigfile.bin
create mode 100644 minio-lab/data1/.minio.sys/buckets/.background-heal.json/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/buckets/.bloomcycle.bin/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/buckets/.usage-cache.bin.bkp/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/buckets/.usage-cache.bin/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/buckets/.usage.json/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/buckets/my-cloud-storage/.metadata.bin/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/buckets/my-cloud-storage/.usage-cache.bin.bkp/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/buckets/my-cloud-storage/.usage-cache.bin/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/config/config.json/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/config/iam/format.json/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/format.json
create mode 100644 minio-lab/data1/.minio.sys/pool.bin/xl.meta
create mode 100644 minio-lab/data1/.minio.sys/tmp/.trash/17764a47-0fa9-4608-bce5-9c23f2cc4ca2/xl.meta.bkp
create mode 100644 minio-lab/data1/.minio.sys/tmp/.trash/1b8afe7e-679a-498b-b589-dd135cdc64aa/xl.meta.bkp
create mode 100644 minio-lab/data1/.minio.sys/tmp/.trash/40c1b6bf-d179-4f34-8465-d1be7ba665e9/xl.meta.bkp
create mode 100644 minio-lab/data1/.minio.sys/tmp/.trash/4eed240d-a429-4f57-a9e1-b2b45c6bdbab/xl.meta.bkp
create mode 100644 minio-lab/data1/.minio.sys/tmp/.trash/5b8fb89a-6e25-4f82-8b79-dec337f1b11a/xl.meta.bkp
create mode 100644 minio-lab/data1/.minio.sys/tmp/.trash/62b18413-fbd8-4894-a729-cde18a7d34c1/xl.meta.bkp

```

Figure 11. Adding and committing the lab files with Git.

```

@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ git commit -m "Complete MinIO distributed storage lab"
create mode 100644 minio-lab/data4/.minio.sys/buckets/my-cloud-storage/.usage-cache.bin/xl.meta
create mode 100644 minio-lab/data4/.minio.sys/config/config.json/xl.meta
create mode 100644 minio-lab/data4/.minio.sys/config/iam/format.json/xl.meta
create mode 100644 minio-lab/data4/.minio.sys/format.json
create mode 100644 minio-lab/data4/.minio.sys/pool.bin/xl.meta
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/13384ec9-cfa8-4f2e-8c93-314f7fe97c0e/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/223aec06-97e9-43ae-b376-21e5cc925b80/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/27d08e2a-e93e-4398-a7b6-6a2f826fbb55/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/50c653f9-0340-432d-ab78-6dbd31c39be7/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/66464350-fd41-4489-9772-3bc624f208d5/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/67dda6d4-0f1d-46e3-a572-e1e42f201e05/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/69f904f7-69cd-49de-9998-a07b52d77a70/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/838cf6ff-d16f-480b-930f-b94da6f199a7/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/98ed0712-3501-4cd2-bc50-edf016207283/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/a4c412c4-5af7-4c6e-a509-fc920d572c8f/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/b131559f-8914-4189-bebe-e002fd863660/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/b2f9b9e0-1ffc-4add-a1fe-4ecf2fee36cc/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/baba46b8-47d3-4ef3-942a-472f0f63d6ce/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/c213e7f3-54c5-4c0e-bb69-ec8c907cdac6/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/fc80b468-9c19-4b91-9418-264cda827e50/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/.trash/fcd17be7-c5d0-4498-aa9a-c1de3b4937ec/xl.meta.bkp
create mode 100644 minio-lab/data4/.minio.sys/tmp/908d5e30-8404-402a-b241-2c74a342f93a
create mode 100644 minio-lab/data4/my-cloud-storage/bigfile.bin/7698c960-2f76-4e72-bc05-139791574227/part.1
create mode 100644 minio-lab/data4/my-cloud-storage/bigfile.bin/7698c960-2f76-4e72-bc05-139791574227/part.2
create mode 100644 minio-lab/data4/my-cloud-storage/bigfile.bin/xl.meta
create mode 100644 minio-lab/docker-compose.yml
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ git push
Enumerating objects: 282, done.
Counting objects: 100% (282/282), done.
Delta compression using up to 2 threads
Compressing objects: 100% (166/166), done.
Writing objects: 100% (281/281), 60.08 MiB | 1.67 MiB/s, done.
Total 281 (delta 67), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (67/67), completed with 1 local object.
To https://github.com/Puffkten-zjq/cloud-computing-lab-github
78da248..b408693 main -> main

```

Figure 12. Completing the commit and pushing the changes to GitHub.

4. Experimental Results and Observations

The experiment was completed successfully. I deployed a MinIO server with four backend storage directories, created a bucket, uploaded a 20 MiB object, observed the internal storage structure, deleted one simulated disk directory, and confirmed that the object was still listed after the failure.

Observation	What I confirmed
Container status	cloud-minio and cloud-mc were both running after docker compose up -d.
Bucket creation	The bucket my-cloud-storage was created successfully.
Object upload	bigfile.bin was uploaded and listed as a 20 MiB standard object.
Backend structure	Each data directory contained object metadata and part files, including xl.meta and part files.
Fault tolerance	After deleting and recreating data3, bigfile.bin was still accessible through mc ls.

5. Reflection

5.1 Problems Encountered and Proposed Solutions

Incorrect directory command at the beginning: At first, the command with spaces around the hyphen was interpreted as different arguments. The shell treated minio, -, and lab separately, and cd reported too many arguments. I fixed it by using minio-lab as one folder name with no spaces.

Accessing the web console in Codespaces: I expected `http://localhost:9001/` to open directly, but Codespaces runs the service in a remote environment. I understood that local browser access may require the Ports panel or port forwarding. Since the lab could be completed from the terminal, I continued with MinIO client commands.

Alias missing after docker compose down: After stopping and recreating the containers, the mc container no longer kept the alias configuration. When I listed the bucket, the alias was not found. I solved this by running `mc alias set` again before checking the object.

Wrong path format for the alias: I also noticed that myminio is an alias, not a Linux path. Therefore, the correct command is `mc ls myminio/my-cloud-storage/` and not a path with a leading slash.

Git path issue: I tried to run `git add minio-lab` while I was already inside minio-lab. Git searched for a nested minio-lab directory and failed. I solved it by returning to the repository root with `cd ..` and then adding minio-lab.

5.2 Why data is still available after deleting one storage directory

The data was still available because MinIO distributed mode stores an object with metadata and erasure-coded parts across multiple storage locations. The object was not stored as one single complete copy only inside data3. When I deleted data3, the remaining storage directories still kept enough information for MinIO to list and serve the object. This is why bigfile.bin still appeared after the cluster was restarted and the alias was configured again. However, this fault tolerance only works within the allowed failure level; if too many storage directories are lost, the object may become unavailable.

6. Conclusion

Through this lab, I built a working four-disk MinIO object storage experiment in Codespaces. I practiced the main workflow of object storage: creating a bucket, uploading an object, listing it through a client, and checking the

backend storage layout. The most useful part for me was observing that the visible object name in the bucket is not the same as a simple local file path. The backend contained metadata and part files distributed across several directories.

The fault-tolerance test also made the idea of distributed storage easier to understand. After I removed data3, the object could still be listed, which showed that MinIO was using redundancy rather than depending on one physical storage location. Overall, the experiment helped me connect Docker Compose, MinIO, S3-style object storage, backend fragmentation, and erasure-coded fault tolerance in one complete process.